

HDOC DAY 33

PROBLEM

Write a C program to perform the following menu-driven operations using separate functions:

1. Read three variable-length 2D integer arrays A, B, and C, each of dimensions (m x n), and verify the associative property of matrix addition: $(A+B)+C=A+(B+C)$
2. Read three variable-length 2D integer arrays with the following dimensions: A (m x n), B (n x p), C (p x q). Verify the associative property of matrix multiplication: $(A \times B) \times C = A \times (B \times C)$
3. Exit

C CODE

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
/* Function prototypes */
```

```
void readMatrix(int **a, int rows, int cols);
```

```
void displayMatrix(int **a, int rows, int cols);
```

```
void addMatrices(int **a, int **b, int **result, int rows, int cols);
```

```
void multiplyMatrices(int **a, int **b, int **result,  
                     int r1, int c1, int c2);
```

```
int compareMatrices(int **a, int **b, int rows, int cols);
```

```
int **allocateMatrix(int rows, int cols);
```

```
void freeMatrix(int **a, int rows);
```

```
int **allocateMatrix(int rows, int cols)
```

```
{
```

```
int **a = (int **)malloc(rows * sizeof(int *));  
  
int i;  
  
for (i = 0; i < rows; i++)  
    a[i] = (int *)malloc(cols * sizeof(int));  
  
return a;  
}
```

```
void freeMatrix(int **a, int rows)  
{  
    int i;  
  
    for (i = 0; i < rows; i++)  
        free(a[i]);  
  
    free(a);  
}
```

```
void readMatrix(int **a, int rows, int cols)  
{  
    int i, j;  
  
    for (i = 0; i < rows; i++)  
    {  
        for (j = 0; j < cols; j++)  
        {  
            scanf("%d", &a[i][j]);  
        }  
    }  
}
```

```
    }  
}  
}
```

```
void displayMatrix(int **a, int rows, int cols)
```

```
{  
    int i, j;  
  
    for (i = 0; i < rows; i++)  
    {  
        for (j = 0; j < cols; j++)  
            printf("%d\t", a[i][j]);  
  
        printf("\n");  
    }  
}
```

```
void addMatrices(int **a, int **b, int **result,  
                int rows, int cols)
```

```
{  
    int i, j;  
  
    for (i = 0; i < rows; i++)  
    {  
        for (j = 0; j < cols; j++)  
            result[i][j] = a[i][j] + b[i][j];  
    }  
}
```

```
void multiplyMatrices(int **a, int **b, int **result,
                     int r1, int c1, int c2)
{
    int i, j, k;

    for (i = 0; i < r1; i++)
    {
        for (j = 0; j < c2; j++)
        {
            result[i][j] = 0;

            for (k = 0; k < c1; k++)
                result[i][j] += a[i][k] * b[k][j];
        }
    }
}
```

```
int compareMatrices(int **a, int **b, int rows, int cols)
{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            if (a[i][j] != b[i][j])
                return 0;
        }
    }
}
```

```

    }
}

return 1;
}

int main()
{
    int choice;

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Verify Associative Property of Matrix Addition\n");
        printf("2. Verify Associative Property of Matrix Multiplication\n");
        printf("3. Exit\n");
        printf("=====\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
            {
                int m, n;

                int **A, **B, **C;

                int **AB, **BC;

                int i;

```

```
printf("\nEnter dimensions (m x n): ");
```

```
scanf("%d %d", &m, &n);
```

```
A = allocateMatrix(m, n);
```

```
B = allocateMatrix(m, n);
```

```
C = allocateMatrix(m, n);
```

```
AB = allocateMatrix(m, n);
```

```
BC = allocateMatrix(m, n);
```

```
printf("Enter elements of matrix A:\n");
```

```
readMatrix(A, m, n);
```

```
printf("Enter elements of matrix B:\n");
```

```
readMatrix(B, m, n);
```

```
printf("Enter elements of matrix C:\n");
```

```
readMatrix(C, m, n);
```

```
/*
```

```
    AB stores (A + B)
```

```
    BC stores (B + C)
```

```
*/
```

```
addMatrices(A, B, AB, m, n);
```

```
addMatrices(B, C, BC, m, n);
```

```
/*
```

Add C to (A+B), and A to (B+C).

The result is stored back in AB and BC.

```
*/  
  
for (i = 0; i < m; i++)  
{  
    int j;  
    for (j = 0; j < n; j++)  
    {  
        AB[i][j] = AB[i][j] + C[i][j];  
        BC[i][j] = BC[i][j] + A[i][j];  
    }  
}  
  
printf("\n(A + B) + C:\n");  
displayMatrix(AB, m, n);  
  
printf("\nA + (B + C):\n");  
displayMatrix(BC, m, n);  
  
if (compareMatrices(AB, BC, m, n))  
    printf("\nMatrix addition is associative.\n");  
else  
    printf("\nMatrix addition is NOT associative.\n");  
  
freeMatrix(A, m);  
freeMatrix(B, m);  
freeMatrix(C, m);  
freeMatrix(AB, m);
```

```

    freeMatrix(BC, m);

    break;
}

case 2:
{
    int m, n, p, q;
    int **A, **B, **C;
    int **AB, **BC;
    int **left, **right;

    printf("\nEnter dimensions:\n");
    printf("Matrix A: m x n\n");
    printf("Matrix B: n x p\n");
    printf("Matrix C: p x q\n");

    printf("Enter m, n, p, q: ");
    scanf("%d %d %d %d", &m, &n, &p, &q);

    A = allocateMatrix(m, n);
    B = allocateMatrix(n, p);
    C = allocateMatrix(p, q);

    /*
    AB = A x B -> m x p
    BC = B x C -> n x q
    left = AB x C -> m x q

```

```

    right = A x BC -> m x q
*/

AB = allocateMatrix(m, p);
BC = allocateMatrix(n, q);
left = allocateMatrix(m, q);
right = allocateMatrix(m, q);

printf("\nEnter elements of matrix A (%d x %d):\n",
    m, n);
readMatrix(A, m, n);

printf("Enter elements of matrix B (%d x %d):\n",
    n, p);
readMatrix(B, n, p);

printf("Enter elements of matrix C (%d x %d):\n",
    p, q);
readMatrix(C, p, q);

/* Calculate (A x B) x C */
multiplyMatrices(A, B, AB, m, n, p);
multiplyMatrices(AB, C, left, m, p, q);

/* Calculate A x (B x C) */
multiplyMatrices(B, C, BC, n, p, q);
multiplyMatrices(A, BC, right, m, n, q);

printf("\n(A x B) x C:\n");

```

```

displayMatrix(left, m, q);

printf("\nA x (B x C):\n");
displayMatrix(right, m, q);

if (compareMatrices(left, right, m, q))
    printf("\nMatrix multiplication is associative.\n");
else
    printf("\nMatrix multiplication is NOT associative.\n");

freeMatrix(A, m);
freeMatrix(B, n);
freeMatrix(C, p);
freeMatrix(AB, m);
freeMatrix(BC, n);
freeMatrix(left, m);
freeMatrix(right, m);

break;
}

case 3:
    printf("\nExiting program...\n");
    break;

default:
    printf("\nInvalid choice! Please try again.\n");
}

```

```
} while (choice != 3);  
  
return 0;  
}
```